CN Lab Project (CS3001)

ClassBoard

LAN-Based Multi-User Collaborative Whiteboard

Technical Documentation and User Guide

Python 3 - raw TCP sockets - length-prefixed JSON protocol - server-sequenced total order - threads - Tkinter. Works with the internet switched off.

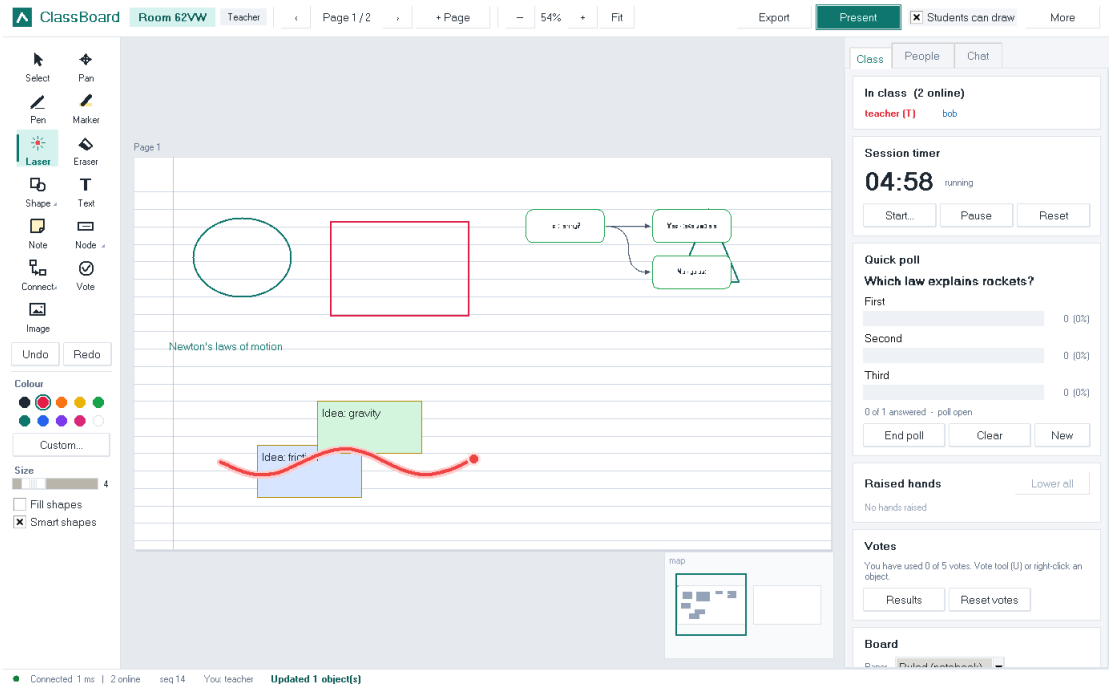


Figure 1 - The teacher's window: lesson pages on ruled paper, a flowchart, sticky notes, the laser pointer, and the Class panel with the timer and a live poll (real screenshot).

Team member	Roll number	Main area
Alahari Tarak Ram	CS24B1031	Server networking, sessions, recovery, persistence
Ponnangatti Bala Murugan	CS24B1039	Client GUI and teacher features
Vallipalli Lokesh Kumar	CS24B1046	Protocol, synchronisation, undo, security
Bellala Parthasaradhi	CS24B1069	Testing, performance, media, exports

Indian Institute of Information Technology, Design and Manufacturing, Kancheepuram - November 2026

Contents

1. Introduction **4**

1.1 Design goals 4

1.2 Requirements traceability 4

2. System architecture **5**

2.1 Code organisation 6

3. Socket communication and message framing **6**

3.1 Why TCP and not UDP 6

3.2 LAN discovery 7

4. Application-level protocol **8**

4.1 Examples 8

5. Sessions and workspace management **8**

6. Synchronisation strategy: server-sequenced total order **10**

6.1 Nothing is overwritten 10

6.2 Why not a CRDT? 10

7. Concurrency handling **10**

8. Ordering, duplicates and missing updates **11**

9. Undo and redo with multiple users **11**

10. Recovery, persistence and reconnection **12**

11. Security and access control **12**

12. Performance optimisations and measured results **14**

13. Additional features **14**

14. Testing **16**

15. Known limitations **16**

16. User guide **17**

16.1 Configure and start the server 17

16.2 Connect clients over the LAN 17

16.3 Create or join a session 17

16.4 Using the application 17

16.5 If something goes wrong 18

17. Screenshots **19**

18. Team contributions **20**

1. Introduction

ClassBoard is a standalone, server-based whiteboard that lets a teacher and a whole class draw, write and discuss on one shared canvas over a Local Area Network (Wi-Fi, Ethernet or a phone hotspot). It needs **no internet connection**: one laptop runs the server, everybody else runs the client and connects to the server's LAN IP address over TCP sockets that we program directly with Python's `socket` module.

Every change a user makes is sent to the server, which validates it, gives it a global sequence number and forwards it to all participants within a few milliseconds. Because every client applies the same operations in the same order with the same code, every screen shows exactly the same board, even with many people editing at once, slow machines, dropped Wi-Fi and duplicated messages.

1.1 Design goals

- **Correctness first:** no lost, duplicated or reordered edits; identical boards on every machine; proved by automated tests that compare board fingerprints.
- **Low latency:** 2-4 ms average delivery on a LAN (measured, section 12).
- **Robustness:** survives disconnects, server restarts, slow or misbehaving clients, malformed messages.
- **Built for teaching:** pages, presentation (follow-me) mode, laser pointer, polls, raised hands, timer, and role-based permissions.
- **No dependencies at runtime**: Python standard library + Tkinter (Pillow optional for PNG/PDF export).

1.2 Requirements traceability

Every mandatory requirement of the project statement and where it is implemented. The full list, including the test that proves each item, is in `docs/Acceptance_Checklist.md`.

Requirement (project statement)	Implementation	Section
Freehand drawing, erase, colours	Pen, marker, eraser, palette, sizes	16
Real-time synchronisation	Sequenced OP broadcast, live previews	6, 12
Server-side coordination	Validate - sequence - apply - broadcast under a room lock	6, 7
Shared, consistent canvas	Deterministic BoardModel.apply on every client	6
Canvas reset and persistence	Host clear (undoable); save/load; autosave; files	10
Session creation and joining	Room codes, passwords, LAN discovery	5, 16
Participant management	Participant list, join/leave notices, roles	11, 16
User identification	User colours, named live cursors, author tags	13
Session continuity, late join	Snapshot + state in WELCOME	5, 10
Operation transmission and broadcasting	TCP framing, writer queues	3, 7
Concurrent editing	Total order; layered move/edit operations	6
Ordering and reliability	seq, op_id de-duplication, ACK, buffering	8
Connection management	Heartbeat, grace period, auto-reconnect	10
Group chat with history	CHAT messages with name and time; history	5, 16
State representation and sync	BoardModel, snapshots, delta sync	6, 10
Recovery after disconnection	Missed ops or snapshot; offline edits kept	10
Undo / redo with multiple users	Per-user stacks, active-flag toggling	9

2. System architecture

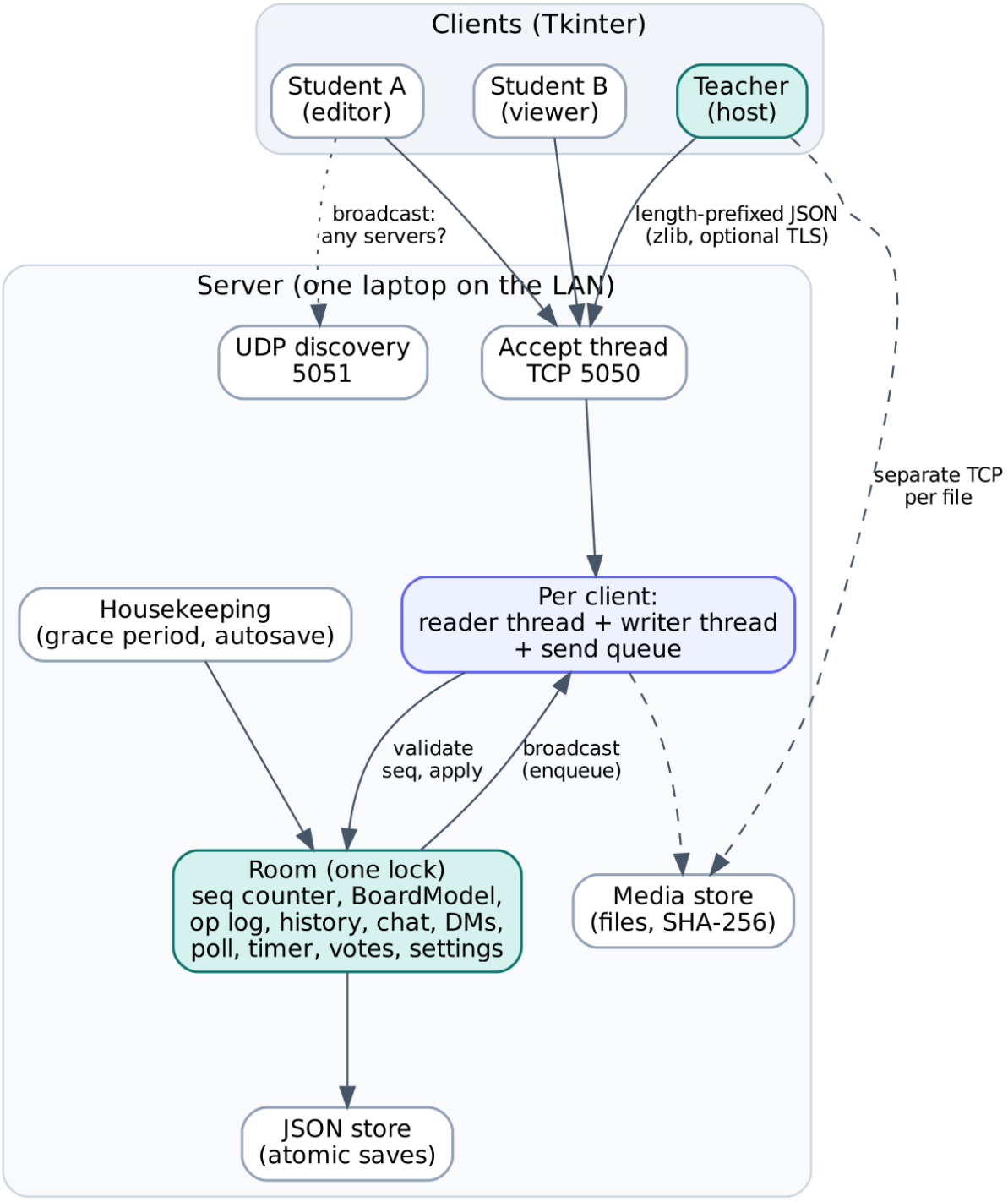


Figure 2 - Server-authoritative star topology. Clients never talk to each other; the server owns the order of all changes.

Server. One accept thread hands every TCP connection to a `ClientHandler` with two threads: a *reader* (receive, validate, dispatch) and a *writer* (drains this client's own send queue). Rooms are independent; each room has one lock that protects its whole state: the sequence counter, the board model, the operation log and history, chat, private messages, votes, poll, timer, settings and participants. A housekeeping thread expires seats of users who did not come back and autosaves rooms every 30 s. A UDP thread answers LAN discovery. An optional Tk dashboard shows rooms and traffic live.

Client. The Tkinter main thread owns every widget. A `NetClient` runs receiver, writer and heartbeat threads and passes incoming messages to the GUI through a thread-safe queue that the Tk loop drains every 15 ms. Large files travel on their own short TCP connections.

2.1 Code organisation

Package / module	Responsibility
common/protocol.py	Message types, framing (length prefix, zlib), validation, TLS helpers
common/model.py	BoardModel: apply(op), visibility, moves, edits, snapshot, fingerprint
common/geometry.py	Pages, flowchart node outlines, connector routing
common/shapes.py	Smart shape recognition (pure geometry, no AI)
server/main.py, handler.py	Accept loop, TLS, per-client threads, dispatch, rate limit
server/room.py	Sessions, sequencing, undo/redo, roles, chat, DMs, polls, history
server/store.py, media.py	Atomic JSON saves and autosave; file uploads/downloads
client/net.py	Connection, ordering, heartbeat, reconnect, pending ops
client/ui/*.py	Start window, main window, board canvas, exports, theme
tests/	110 automated tests incl. 27 that drive the real GUI

3. Socket communication and message framing

The server creates a TCP socket, binds it to `0.0.0.0:5050` (all interfaces, so Wi-Fi, Ethernet and hotspot clients can all reach it) and listens. It prints its LAN addresses at start-up. Clients call `socket.create_connection((server_ip, 5050))`. Both sides set `TCP_NODELAY` (turns off Nagle's algorithm so small drawing messages leave immediately) and `SO_KEEPALIVE`.

Framing. TCP delivers a byte stream, not messages: one `recv()` may return half a message, or several messages glued together. Every message is therefore sent as a frame:

```
+-----------------------------+-----------------------------------------+
| length: 4-byte unsigned int  | body: UTF-8 JSON object (length bytes)    |
| big-endian (network order)   | {"type": "STROKE", "op_id": ..., ...}     |
+-----------------------------+-----------------------------------------+
  top bit of the length = 1  ->  body is zlib-compressed (used above 32 KB)
```

- `recv_exact(n)` loops until exactly n bytes have arrived, which handles fragmentation; reading header then body handles coalescing. An incremental `FrameDecoder` is tested by feeding a stream one byte at a time.
- Frames larger than 4 MB on the wire are rejected *before* allocating memory; compressed frames may expand to at most 32 MB (protects against 'zip bombs').
- Big frames (board snapshots, history) are compressed with zlib: a 2,000-object snapshot shrinks from 482 KB to 70 KB (-85%).
- Files (images, video, voice) use a **separate short-lived TCP connection** per transfer with a JSON header followed by raw bytes and a SHA-256 check, so a large video never blocks drawing traffic (head-of-line blocking).

3.1 Why TCP and not UDP

A whiteboard cannot tolerate lost or reordered updates: a missing erase or an undo applied before the stroke it undoes would leave screens permanently different. TCP provides reliable, ordered, connection-oriented delivery and flow control; with UDP we would have to re-implement acknowledgements, retransmission, ordering and fragmentation (snapshots are far bigger than one

datagram). On a LAN, TCP's extra cost is about a millisecond. The accepted trade-off is head-of-line blocking, which we avoid for large files by using separate connections. UDP is used only for discovery, where broadcast is required and a lost packet is harmless.

3.2 LAN discovery

The client broadcasts a small UDP datagram `WB_DISCOVER` to port 5051 (on 255.255.255.255 and each subnet's directed broadcast). Every server answers by unicast with `WB_HERE`: host name, TCP port, protocol version, whether TLS is on, and its rooms. Users can still type the IP address when a network blocks broadcasts.

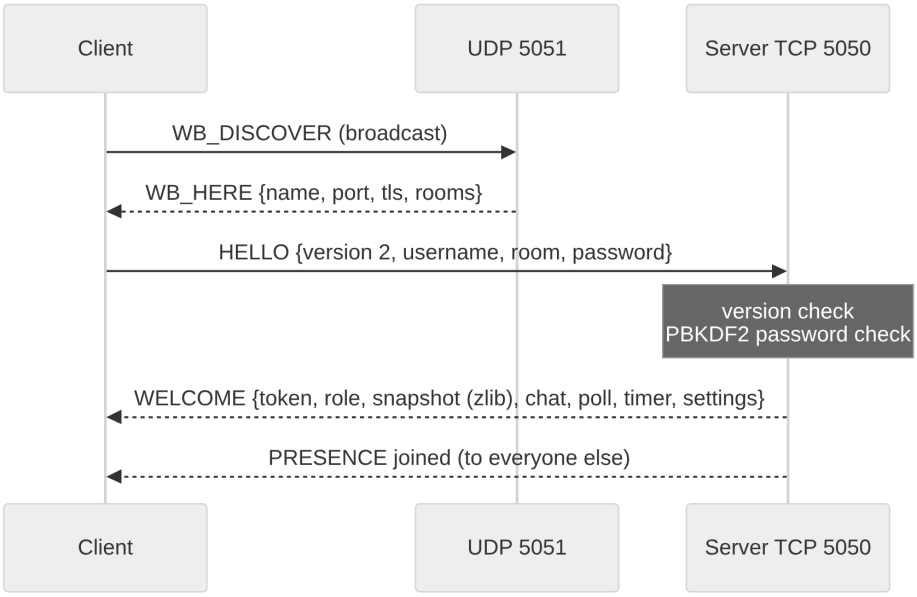


Figure 3 - Discovering a server and joining a room.

4. Application-level protocol

Every message is a JSON object with a `type`. Board operations carry a client-generated `op_id` (unique per client) used for acknowledgement and duplicate detection; the server adds `seq` (global order), `user` and `ts` before broadcasting. The protocol is versioned (`version: 2` in HELLO) and mismatching clients receive a clear error.

Group	Client to server	Server to client
Session	HELLO (create / join / reconnect), PING, BYE, CLOSE_ROOM	WELCOME (full state or missed ops), PONG, PRESENCE, KICKED, ERROR, INFO
Board (sequenced)	STROKE, MOVE, EDIT, ERASE, CLEAR, UNDO, REDO	OP {seq, op} to everyone (echo = acknowledgement), ACK (no-op / duplicate)
Live, not stored	STROKE_PART (preview), CURSOR, LASER	PREVIEW, CURSOR, LASER
Communication	CHAT, DM (private)	CHAT, DM
Classroom	VOTE, VOTES_CLEAR, TIMER, POLL, HAND, FOLLOW, VIEWPORT, SETTINGS	VOTES, TIMER_STATE, POLL_STATE, PRESENCE(hand), ROOM_STATE, VIEWPORT, SETTINGS
Admin / persistence	ADMIN (lock, roles, kick), SAVE, LOAD, LIST_SAVES, RESTORE, HISTORY	SAVES, RESET (board replaced), HISTORY
Files (own connection)	MEDIA_PUT + raw bytes, MEDIA_GET	MEDIA_READY, MEDIA_OK, MEDIA_DATA + raw bytes

4.1 Examples

```
client -> server
{"type":"STROKE","op_id":"3fa9c1-17","stroke":{"id":"3fa9c1-s16","tool":"pen",
 "color":"#e11d48","width":4,"points":[10,20,12,24,15,30]}}

server -> every client (including the sender)
{"type":"OP","seq":118,"op":{"kind":"STROKE","op_id":"3fa9c1-17","user":"asha",
 "seq":118,"ts":1731400000.12,"stroke":{...}}}

move and edit an existing object
{"type":"MOVE","op_id":"3fa9c1-18","stroke_ids":["3fa9c1-s16"],"dx":40,"dy":0}
{"type":"EDIT","op_id":"3fa9c1-19","stroke_id":"3fa9c1-s16","set":{"color":"#0f766e"}}
```

One object format. Every board object is a *stroke* with a `tool`: pen, highlighter, line, arrow, rect, ellipse, polygon (triangle, diamond), text, sticky, image, node (flowchart) and connector (with `src`/`dst` node ids). The server validates one format with whitelisted fields and bounds, and never needs to understand drawing.

5. Sessions and workspace management

- **Create:** the first HELLO with `create: true` makes a room with a random 4-character code (no ambiguous 0/O, 1/I) and an optional password; the creator becomes host (shown as *Teacher*).
- **Join:** HELLO with the room code (+ password). The server answers WELCOME containing a session token, role and colour, the participant list, a snapshot of the board, the chat history, the presenter's view, poll, timer and settings, so a late joiner sees exactly the current workspace.
- **Participants:** PRESENCE messages announce joined, left, away (connection lost), reconnected, role changes and raised hands; the GUI shows a participant list with colours and a chat notice for each event.
- **Continuity:** users can join or leave at any time; nothing pauses for the others. If the host leaves, the longest-present participant becomes host.

- **Roles:** host (teacher), editor (student) and viewer (view-only student), changeable live; kicking bans the name from that room; the host can end the session for everyone.

6. Synchronisation strategy: server-sequenced total order

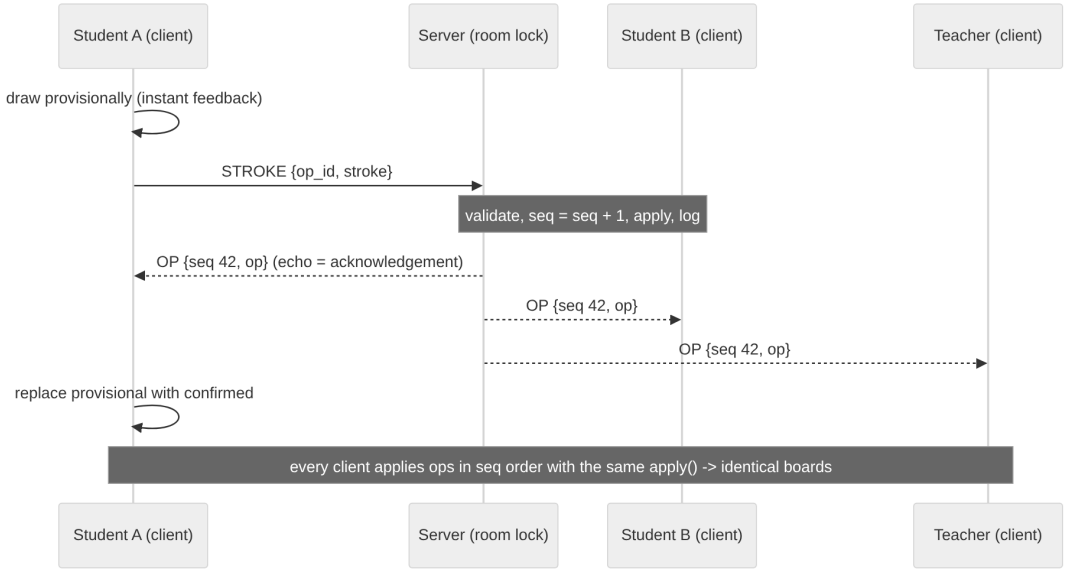


Figure 4 - One drawing operation. The sender's own echo is its acknowledgement.

Under the room lock the server performs **validate, seq = seq + 1, apply, log, enqueue to every client** as one atomic step. Clients apply OP messages strictly in `seq` order using `BoardModel.apply()` from `common/model.py`, the very same code the server runs. Same initial state + same operations + same order + deterministic function = the same board everywhere.

6.1 Nothing is overwritten

Objects are never changed in place or deleted. Their visible state is derived:

```
visible(o)   = creating op active  AND  no active ERASE/CLEAR covers o
position(o)  = stored points + sum of (dx, dy) of active MOVE ops on o
look(o)      = stored fields  <-  every active EDIT on o, in seq order
```

Therefore undo/redo only flips an operation's `active` flag, and concurrent operations combine deterministically:

- Two users draw at the same time: different objects with unique ids, both kept.
- Two users drag or recolour the same sticky note: both ops are applied in seq order; the later one wins identically on every screen.
- A user erases an object another user is moving: the erase hides it everywhere; the late move is harmless; undoing the erase brings it back.
- A flowchart node is erased: connectors attached to it are hidden too and return with an undo, because connectors are drawn from their nodes' current positions.

6.2 Why not a CRDT?

A CRDT (for example last-writer-wins registers with Lamport clocks) converges without any central order and suits peer-to-peer or offline-first systems. A LAN classroom always has a server, so a single sequencer gives the same convergence with much less machinery and a simple proof; it also makes undo, version history (replaying the log) and restore straightforward. The cost is that edits need a round trip to the server, which we hide with optimistic (provisional) drawing.

7. Concurrency handling

- **One lock per room** (an RLock) protects validate-seq-apply-log-enqueue, so requests from different reader threads are serialised into one total order without lost updates.
- **No network I/O while holding a lock.** Broadcasting only puts the encoded frame on each client's queue; the per-client writer thread does the slow `sendall`. A frozen client therefore cannot stall the room; if its queue exceeds 5,000 frames it is disconnected (it can reconnect and catch up).
- **Lock order** is fixed (room-manager lock, then a room lock), so deadlock is impossible.
- **Rate limiting** by a token bucket per connection (300 msg/s, bursts of 600): a flooding client's own reader thread is paused, so TCP flow control slows that sender down while nothing is dropped.
- **Client side:** only the Tk thread touches widgets; network threads communicate through a queue drained every 15 ms (at most 300 events per tick), so the interface never freezes.
- **Optimistic rendering:** your own stroke appears instantly as a provisional drawing and is replaced by the confirmed one when its OP echo arrives, or rolled back if the server refuses it (e.g. the board was locked).

8. Ordering, duplicates and missing updates

Problem	How it is handled
Out-of-order delivery	Clients buffer an OP whose seq is not the next one and deliver contiguous runs only.
Duplicates	The server remembers recent op_ids: a repeated op is not re-applied and gets an ACK. Clients ignore any seq not greater than the last applied.
Missing updates after a drop	On reconnect the client sends its last_seq; the server returns exactly the ops after it (or a snapshot).
Lost sends	Board ops stay 'pending' until their echo arrives and are re-sent after a reconnect with the same op_id, so each is applied exactly once.
Malformed or hostile input	Whitelisted fields and bounds; three bad messages disconnect only that client; fuzz-tested.

9. Undo and redo with multiple users

The server keeps an undo and a redo stack **per user**. Ctrl+Z sends UNDO; the server pops the user's most recent still-active operation, marks it inactive and broadcasts a sequenced UNDO op that every client applies. Rules:

- You only ever undo your own work, never someone else's.
- Undoing a stroke hides it; undoing an erase or clear brings objects back; undoing a move or edit returns the previous position or look.
- An operation that has no effect any more is skipped.
- Any new action clears your redo stack; simultaneous undos are serialised by the room lock like every other operation.
- Clearing the board (host only) is itself undoable.

10. Recovery, persistence and reconnection

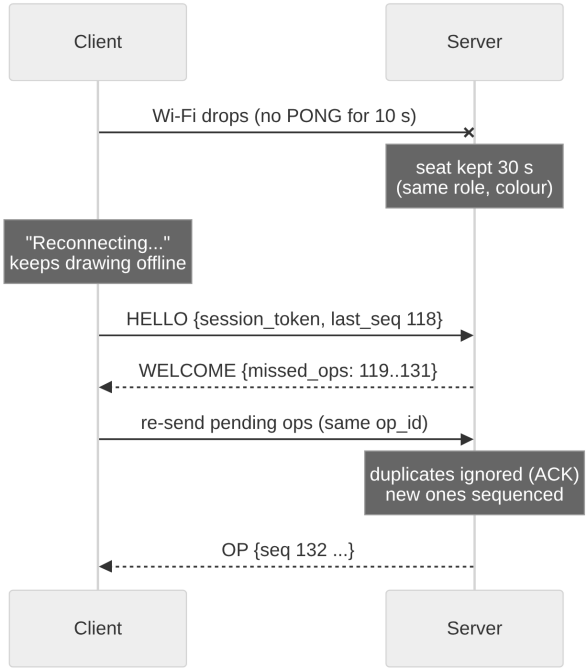


Figure 5 - Reconnecting after a Wi-Fi drop.

- **Detecting failure:** the client sends PING every 3 s (and shows the round-trip time); 10 s without data means the connection is dead on either side. This catches Wi-Fi drops that TCP alone would notice only after minutes.
- **Grace period:** the server keeps a dropped user's seat (name, role, colour, undo history) for 30 s and shows them as 'away'.
- **Reconnect:** exponential backoff 1, 2, 4, 8 s; HELLO carries the session token and last_seq; the server replies with only the missed operations, or a snapshot when the gap is too old. Pending operations are re-sent.
- **Long outage:** after the grace period the client simply rejoins under the same name, receives a snapshot, and its offline edits are still applied.
- **Server restart:** every room is autosaved every 30 s and on shutdown, written atomically (temporary file then rename) and restored at start-up; clients keep retrying and reconnect by themselves.
- **Saving:** the teacher can save named versions on the server, load them later, save a board file on their own laptop and open it in another class, or restore any earlier version from the version history (the server keeps the operation log; the client replays it with the same apply()).

11. Security and access control

Threat / need	Measure
Password disclosure	PBKDF2-HMAC-SHA256, 120,000 iterations, random salt per room; constant-time comparison
Password guessing	3 failures per minute block that IP for 30 s
Modified client bypassing the UI	Every permission is checked on the server for every message (roles, lock, own-work-only, host-only actions)
Malformed / oversized data	Whitelisted fields, numeric bounds, NaN and type checks, 4 MB frame limit, 32 MB inflate limit, list and text limits

Threat / need	Measure
Flooding	Per-connection token bucket; slow clients isolated by their own queue
Eavesdropping	Plain JSON by default (trusted lab LAN); optional TLS 1.2+ with --tls-cert/--tls-key for session and file connections
Privacy	Private messages delivered only to the two people; poll answers anonymous to students
Version mismatch	Protocol version checked in HELLO, with a clear error

Roles: the **teacher** (host) can clear, lock, change settings, run polls and the timer, present, save/load/restore, change roles, remove students and end the session; **students** (editors) can draw; **view-only students** can watch, chat, vote, answer polls and raise a hand. *Own-work-only* mode restricts editors to the objects they created.

12. Performance optimisations and measured results

Measured with `tools/benchmark.py` on one laptop (all clients in one process, so real LAN numbers are similar or better). Latency is the time from a client sending an operation until another client has received it.

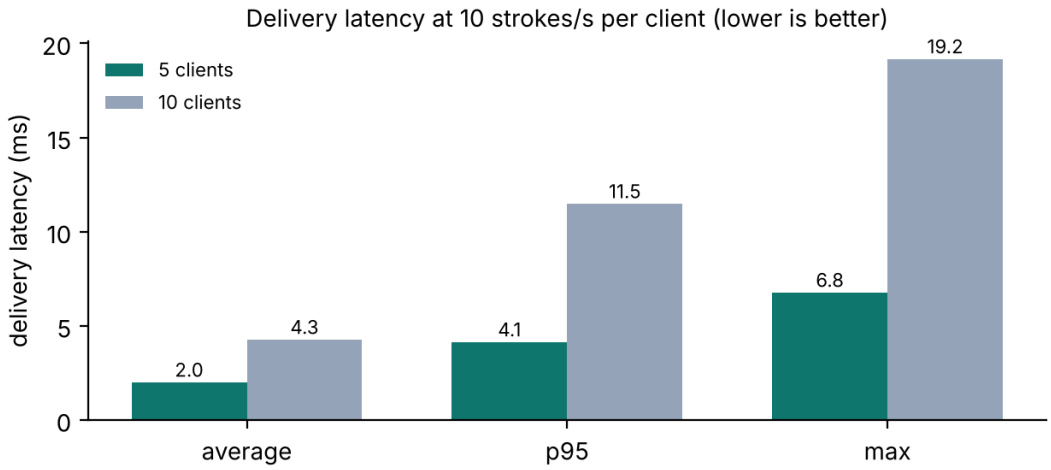


Figure 6 - Delivery latency with every client drawing 10 strokes per second.

Test	Result
5 clients, 10 strokes/s each	avg 2.0 ms, p95 4.14 ms, boards identical
10 clients, 10 strokes/s each	avg 4.27 ms, p95 11.48 ms, boards identical
Burst, 5 clients x 200 strokes	2222 ops/s sequenced, 11110 deliveries/s
Burst, 10 clients x 200 strokes	1020 ops/s sequenced, 10203 deliveries/s
Late-join snapshot (2,000 objects)	482 KB JSON -> 70 KB on the wire (-85.4%)

- Live stroke previews are streamed incrementally at 20 Hz instead of per mouse event; sub-pixel mouse moves are dropped; cursors are throttled.
- Each broadcast is encoded once and the same bytes are queued to every client.
- Delta synchronisation on reconnect; zlib compression for large frames.
- Separate connections for files keep drawing latency flat during big transfers (0.4 ms median during a 60 MB upload).
- The canvas moves existing items when panning or zooming and redraws crisply after a short pause, so large boards stay smooth.

13. Additional features

Beyond the mandatory requirements (the project statement encourages extensions):

Feature	Description
Pages and paper	Lesson pages on plain, grid, ruled, dot or graph paper; page navigation
Presentation (follow-me)	Students' views follow the teacher's page, pan and zoom
Laser pointer	Fading red trail seen by everyone, not stored
Flowcharts and mind maps	5 node shapes, labelled connectors (arrow, line, curve); Tab = child, Enter = sibling
Sticky notes, dot voting	5 votes per person, badges, live results ranking
Quick polls, raised hands, timer	Live poll results; teacher sees answers; shared countdown

Feature	Description
Selection and editing	Move, resize, recolour, duplicate, delete, edit text in place
Smart shapes	Rough circles, rectangles, triangles and lines become perfect shapes
Infinite canvas + minimap	Zoom 5%-800%; minimap with everyone's cursors
Contributor identification	Hover shows creator and last editor; author tags
Version history	Slider and replay of the whole lesson; restore any version
Media sharing	Images on the board; images, video, audio, voice messages and files in chat
Private messages	One chat tab per conversation
Export	Page or board as PNG, all pages as PDF notes, SVG, board files
Server dashboard, TLS	Live rooms and traffic window; optional encryption

14. Testing

`python -m pytest tests -q` runs **110 automated tests**; they pass on Python 3.10, 3.12 and 3.13.

- **Framing and protocol:** byte-by-byte and coalesced streams, oversized, invalid and compressed frames, validation of every object type.
- **Consistency:** 10 clients x 200 simultaneous strokes, concurrent moves and edits by 5 clients; every client's board fingerprint must be identical.
- **Reliability:** late join, reconnect delta, offline operations delivered exactly once, heartbeat timeout, rejoin after the grace period, a full server restart in the middle of a class.
- **Security:** passwords and rate limiting, version mismatch, roles, lock, kick, own-work-only, TLS, ~2,000 random malformed messages (fuzzing), a flooding client.
- **Real GUI (27 tests):** a teacher window and a student window connected through a real server are driven with mouse events; each test checks the result on the other user's screen (tools, selection, flowcharts, presentation mode, polls, chat, export, history, reconnect, hand-over, end session).
- **Executables:** built with PyInstaller and run in a network namespace with no internet access.

15. Known limitations

- Traffic is plaintext unless TLS is enabled; a self-signed certificate encrypts but does not authenticate the server unless it is distributed to the clients.
- One thread pair per client: ideal for a classroom (tens of users); hundreds of users would call for selectors/asyncio.
- Version history is kept in server memory and starts again after a restart (the board itself is restored).
- The eraser removes whole objects (no pixel erasing).

16. User guide

16.1 Configure and start the server

- Run `ClassBoard-Server.exe` (or `python run_server.py`, or double-click `start_server.bat` for the dashboard) on one laptop.
- It prints the address students should use, e.g. *Clients can connect to: 192.168.43.17*. Default ports: TCP 5050, UDP 5051.
- Windows firewall: allow access on **Private** networks, or run `tools\open_firewall_windows.bat` as administrator; make sure the Wi-Fi/hotspot is a Private network.
- Options: `--port`, `--udp-port`, `--save-dir`, `--dashboard`, `--stats 10`, `--tls-cert/--tls-key`, `--no-discovery`. Stop with Ctrl+C (rooms are saved).

16.2 Connect clients over the LAN

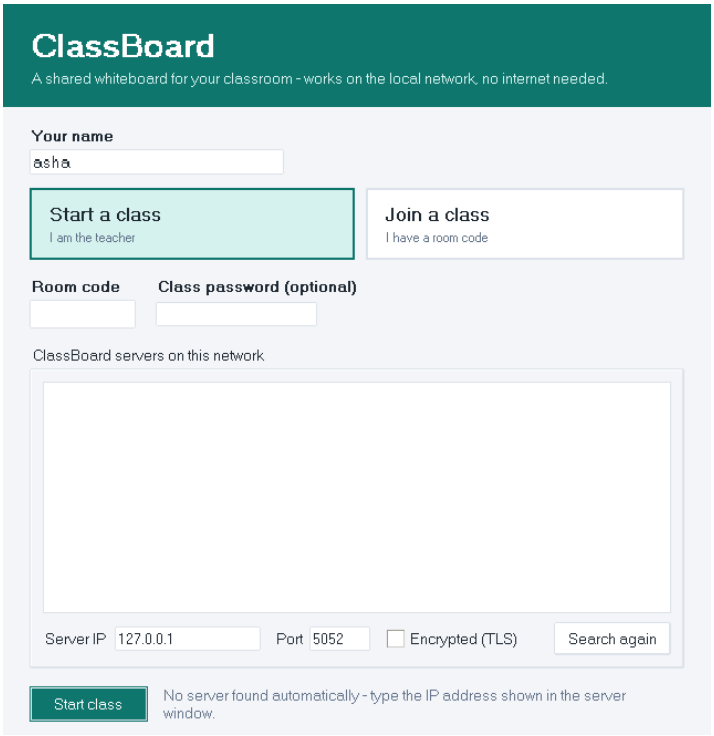


Figure 7 - Start window: name, teacher or student, servers found on the network.

- Start `ClassBoard-Client.exe` on each laptop connected to the same network.
- Type your name. Servers and their classes are found automatically; click one, or type the server IP and port. Tick *Encrypted (TLS)* only if the server was started with TLS (it is ticked automatically when discovered).

16.3 Create or join a session

- **Teacher:** choose *Start a class*, optionally set a class password, click *Start class*. The room code appears at the top of the window (click it to copy).
- **Students:** choose *Join a class*, enter the room code and password, click *Join class* - or click the class in the discovered list.

16.4 Using the application

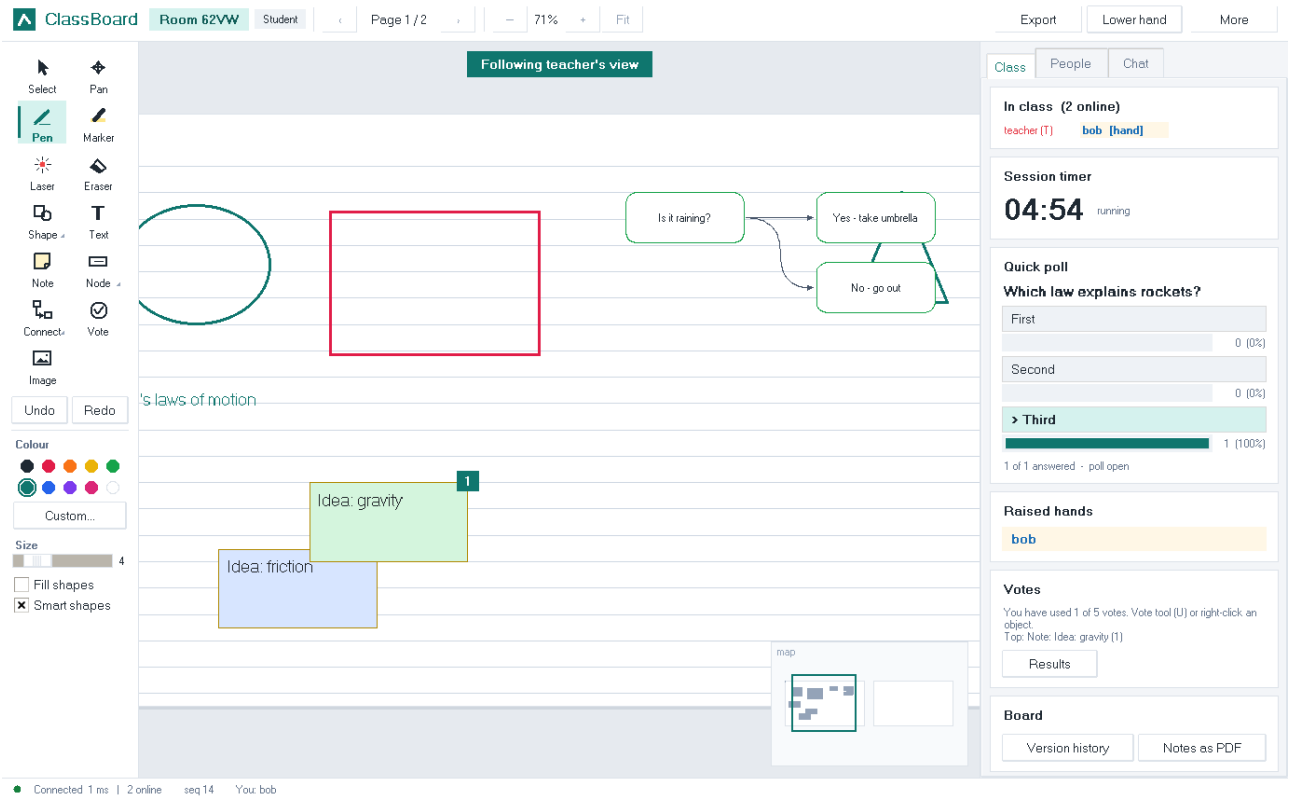


Figure 8 - A student's window while the teacher presents: the view follows the teacher; the poll and raised hand are in the Class panel.

Area	What you can do
Tool rail (left)	Select (V), Pan (H), Pen (P), Marker (M), Laser (K), Eraser (E), Shape (S), Text (T), Note (N), Node (F), Connect (C), Vote (U), Image (I); Undo/Redo; colours, size, fill, smart shapes
Header	Room code, role, pages (PageUp/PageDown, + Page for the teacher), zoom and Fit, Present and Students-can-draw (teacher) or Raise hand (student), Export, More (history, board files, authors, help)
Class tab	Who is in class, timer, quick poll, raised hands, votes, board settings (paper, own-work-only, save/load, open file, clear, history, end session)
People tab	Participants; right-click to message, make view-only, allow drawing, make teacher or remove
Chat tab	Group chat with history and timestamps, private conversations, files, images, video and voice messages
Keyboard	Ctrl+Z / Ctrl+Y undo/redo, Del delete, Ctrl+D duplicate, Ctrl+A select all, Ctrl+V paste, Ctrl+S save board file, Ctrl+E export page, Ctrl+wheel zoom, F1 help

16.5 If something goes wrong

Problem	Fix
No server found	Same network? Type the IP printed by the server; check the firewall (Private network)
Cannot connect	Correct IP and port? Server running? TLS tick must match the server
Port already in use	Another server is running: close it or use --port 5055
Versions differ	Use the same release of server and client everywhere
Connection lost	Keep working - the client reconnects and catches up automatically

17. Screenshots

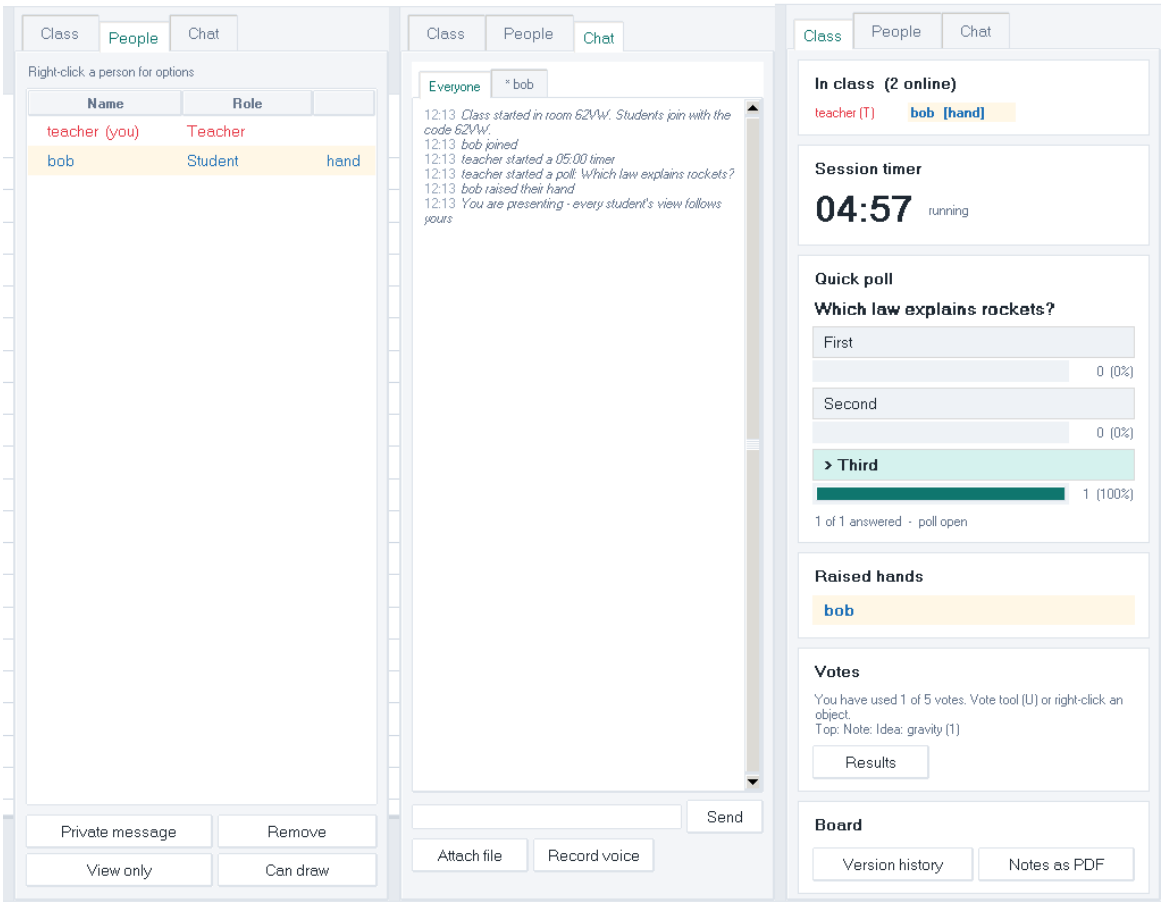


Figure 9 - Teacher's People tab (raised hand), teacher's chat with a private conversation, and a student's Class tab after answering the poll.

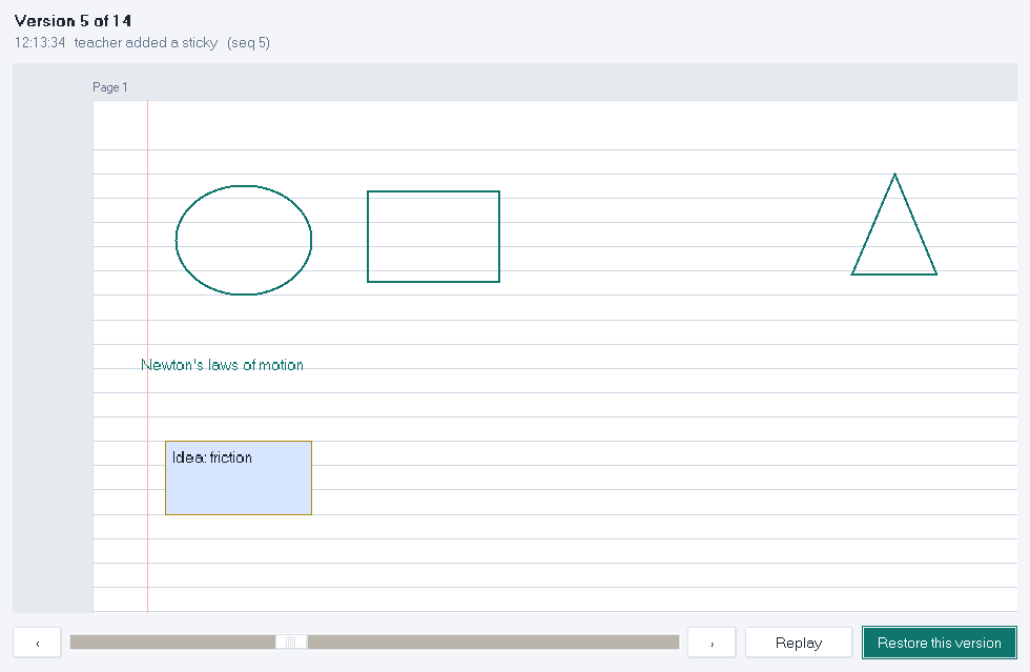


Figure 10 - Version history: slide through every change of the lesson, replay it, or restore a version.



Figure 11 - Server dashboard: rooms, participants and live traffic.

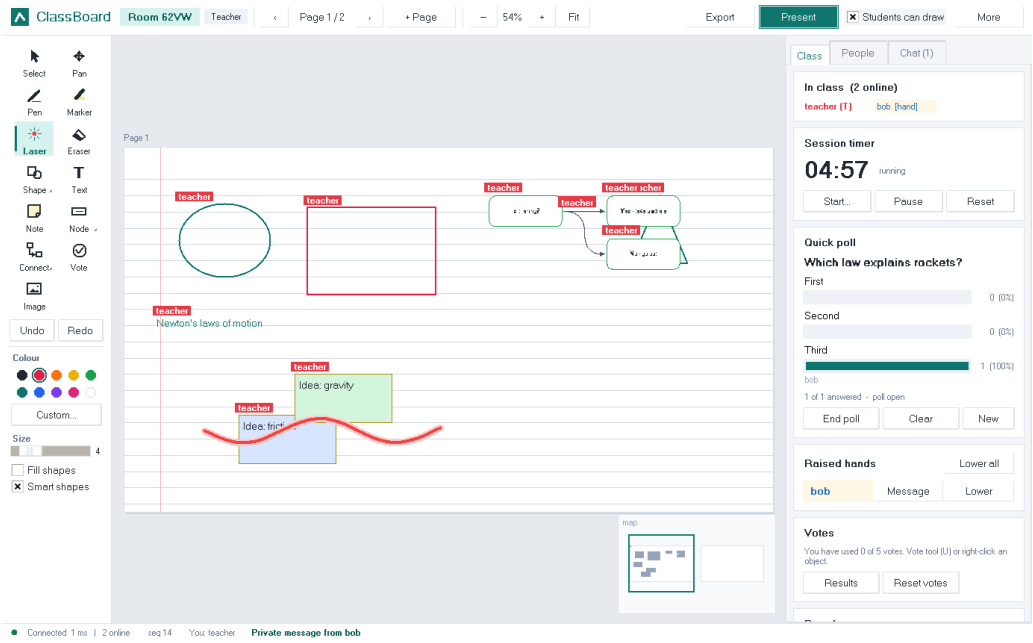


Figure 12 - 'Show authors' tags every object with its creator.

18. Team contributions

Member	Roll number	Contribution
Alahari Tarak Ram	CS24B1031	Server networking, sessions, recovery, persistence
Ponnangatti Bala Murugan	CS24B1039	Client GUI and teacher features
Vallipalli Lokesh Kumar	CS24B1046	Protocol, synchronisation, undo, security
Bellala Parthasaradhi	CS24B1069	Testing, performance, media, exports